UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTAD DE INGENIERÍA

CURSO: 1AMA0726 – MATEMÁTICA COMPUTACIONAL

TÍTULO: PROBLEMA DEL CAMINO MÍNIMO: IMPLEMENTACIÓN INTERACTIVA DEL ALGORITMO DE DIJKSTRA EN GRAFOS DIRIGIDOS ACÍCLICOS

INTEGRANTES	Código
Manuel Herrera	U202510330
Gengis Cam Pizarro	U202517715
Santiago Ceron	U202514848
Nicolas Nima	U202511947
Piero Bobadilla	U202521533

PROFESOR

Antonio Marcos Medina Martínez

UPC, octubre de 2026

Índice de contenido

Introducción 3
Objetivo 3
Metodología 3
Algoritmo de Dijkstra y Grafos Dirigidos Acíclicos 3
Generación de Grafos con Restricciones 4
Desarrollo del código 4
Librerías Utilizadas 4
Descripción General del Programa 5
Ingreso y Configuración del Grafo 5
Modo de Dibujo Interactivo 5
Búsqueda y Selección de Nodos 5
Ejecución Interactiva del Algoritmo 6
Diagrama de flujo del programa ejecutable 6
Visualización de Resultados 7
Programa 7
Resultados 7
Conclusiones 8
Referencias Bibliográficas 9

Problema del camino mínimo: Algoritmo de Dijkstra

Introducción

En el ámbito de la ingeniería y la computación, el estudio de redes y trayectorias mínimas es fundamental para resolver diversos problemas de la vida real, con amplias aplicaciones en cartografía digital, telecomunicaciones y optimización de recursos logísticos (Ahuja et al., 1993; Bondy & Murty, 2008). Desde la formulación inicial de la teoría de grafos por Leonard Euler en 1736 al resolver el célebre problema de los puentes de Königsberg, los modelos basados en redes permitieron formalizar problemas de conectividad e infraestructura. Encontrar el camino más corto permite optimizar tiempo, recursos y energía mediante procedimientos estructurados que facilitan el análisis computacional (Cormen et al., 2022). Por esta razón, el análisis de trayectorias mínimas es uno de los temas centrales de la matemática computacional.

El problema del camino mínimo consiste en encontrar, entre dos nodos de un grafo ponderado, una ruta cuya suma de pesos sea mínima (Ahuja et al., 1993). Al trabajar sobre un grafo dirigido acíclico, la red no contiene ciclos dirigidos, por lo que sus vértices pueden disponerse en un orden topológico en el que toda arista dirigida conecta un vértice anterior con uno posterior (Gross & Yellen, 2005; Kahn, 1962). En grafos con pesos no negativos, el algoritmo de Dijkstra construye las distancias mínimas de forma iterativa: selecciona el nodo no visitado con menor distancia tentativa y relaja sus aristas salientes (Cormen et al., 2022; Dijkstra, 1959). El presente informe expone el desarrollo del proyecto en cuatro componentes clave: marco metodológico, implementación en software y librerías, ejecución interactiva paso a paso con diagrama de flujo, y análisis de resultados con conclusiones.

Objetivo

Objetivo General: Desarrollar una aplicación en Python con una interfaz amigable e interactiva que permita al usuario participar en cada etapa de la ejecución del algoritmo de Dijkstra sobre grafos dirigidos acíclicos de n nodos, con $n \in [7, 16]$, permitiendo generar el grafo de forma aleatoria con restricciones configurables, mediante un formulario de aristas o dibujándolo directamente sobre la pantalla, validando que no existan ciclos, mostrando el procedimiento paso a paso con su tabla de distancias y presentando al final la distancia mínima, la cantidad de caminos mínimos encontrados y la secuencia de nodos de cada uno resaltados en la pantalla gráfica.

Metodología

Algoritmo de Dijkstra y Grafos Dirigidos Acíclicos

Un grafo dirigido ponderado se representa mediante un conjunto de nodos V (donde la cantidad n está entre 7 y 16, identificados con letras desde la A hasta la P), un conjunto de aristas dirigidas E y un conjunto de pesos positivos w(u, v) mayores a cero asignados a cada conexión. Un grafo dirigido acíclico no contiene ciclos dirigidos; sus vértices pueden disponerse en un orden topológico en el que toda arista dirigida conecta un vértice anterior con uno posterior (Gross & Yellen, 2005; Kahn, 1962). Para la visualización, el programa agrupa los nodos en generaciones topológicas, de modo que los ancestros aparecen en generaciones anteriores y los descendientes en generaciones posteriores (NetworkX Developers, 2026a).

El algoritmo de Dijkstra funciona eligiendo en cada iteración el nodo que tenga la menor distancia acumulada provisional desde el origen, fijándolo como visitado o definitivo (Cormen et al., 2022). A continuación, revisa todos sus nodos vecinos salientes y aplica el proceso de relajación: si la distancia del nodo actual más el peso de la arista es menor que la distancia que tenía guardada el vecino, se actualiza con este nuevo valor más bajo y se guarda el nodo actual como su predecesor. Conforme a las pautas de la asignatura en la Lectura 5 de la UPC (Universidad Peruana de Ciencias Aplicadas, s. f.), cada nodo mantiene un seguimiento con su terna de etiquetado [dv, pv, k], donde dv representa la distancia acumulada, pv el vértice predecesor y k la iteración en que se consolidó la etiqueta permanente.

En cada iteración se examinan las aristas salientes del nodo seleccionado. Si dist[u] + w(u, v) < dist[v], se actualizan la distancia tentativa de v y su predecesor; si ambos valores son iguales, el programa conserva todos los predecesores que conducen a una ruta mínima (Cormen et al., 2022; NetworkX Developers, 2026b). De esta manera, al terminar el algoritmo, una función de búsqueda hacia atrás (backtracking) recorre estos predecesores desde el destino hasta el origen para reconstruir todas las rutas mínimas posibles sin omitir ninguna solución, manteniendo una complejidad temporal de O((|V| + |E|) log |V|) gracias al uso de colas de prioridad (Sedgewick & Wayne, 2011; Skiena, 2020).

Generación de Grafos con Restricciones

Para evitar grafos aleatorios triviales, el generador fija primero el origen y el destino y reparte los demás nodos en capas topológicas. Cada nodo recibe al menos una arista de la capa anterior y emite al menos una hacia la siguiente, y se limita a un máximo de tres aristas salientes por nodo, como ocurre en redes reales. Luego, una búsqueda en anchura (BFS) mide la menor cantidad de aristas entre origen y destino y se eliminan atajos hasta que no exista ningún camino de menos de k aristas (k = 4 por defecto). Opcionalmente se fuerza un empate: se calcula la holgura de cada arista, d(s, u) + w(u, v) + d(v, t) − d*, y se reduce el peso de una arista en esa holgura, de modo que aparezca un segundo camino de costo d* sin alterar el óptimo, pues en un DAG los costos d(s, u) y d(v, t) no dependen de dicha arista (Cormen et al., 2022).

Desarrollo del código

Librerías Utilizadas

Para la creación del programa se utilizaron las siguientes librerías de Python:

Tkinter y ttk se utilizaron para implementar la interfaz gráfica de escritorio, los diálogos y el guardado y carga de archivos (Python Software Foundation, 2026b).

Matplotlib se integró en la interfaz Tkinter para dibujar nodos, aristas, flechas y etiquetas de peso, y para detectar los gestos del mouse del modo de dibujo (Matplotlib Development Team, 2026).

Heapq proporcionó el min-heap empleado como cola de prioridad eficiente para extraer el elemento con menor valor tentativo (Python Software Foundation, 2026a).

NetworkX figura como dependencia del proyecto; el cálculo de generaciones topológicas con que se ubican los nodos por capas está implementado en graph_model.py bajo el mismo principio de la función topological_generations (NetworkX Developers, 2026a).

Json, Collections, Math, Random y String se usaron para guardar y cargar grafos, manejar colas de recorrido, representar el valor infinito, generar pesos aleatorios y nombrar los nodos con letras de la A a la P.

Figura 1

Importaciones reales del código fuente del proyecto



Nota. Líneas de importación extraídas de los archivos del programa, con el módulo donde se encuentra cada una. Elaboración propia.

Descripción General del Programa

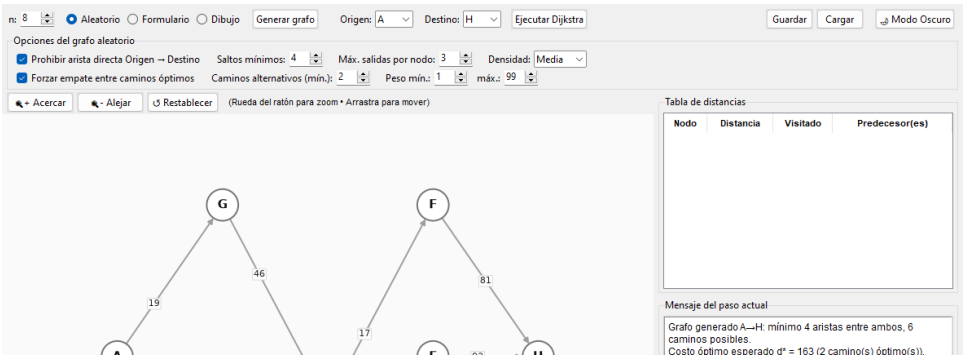
La interfaz se organiza en una barra superior (n, modo de creación, origen, destino, ejecución, guardado y carga, y modo oscuro), un panel de opciones del grafo aleatorio, el lienzo del grafo a la izquierda y el panel de información a la derecha. El grafo puede obtenerse de tres formas: aleatoria con restricciones, mediante un formulario de aristas o dibujándolo con el mouse; además puede guardarse y cargarse en formato JSON.

Ingreso y Configuración del Grafo

El usuario define «n» entre 7 y 16 nodos y elige el modo de creación. En el modo aleatorio selecciona el origen y el destino antes de generar y configura los saltos mínimos, el máximo de salidas por nodo, la densidad, la prohibición de la arista directa, el empate forzado, la cantidad mínima de caminos alternativos y el rango de pesos. En el modo de formulario se añaden aristas indicando origen, destino y peso; en ambos modos manuales, una búsqueda en profundidad (DFS) detecta si la nueva arista crearía un ciclo y, de ser así, la rechaza con una alerta (Cormen et al., 2022; Skiena, 2020).

Figura 2

Modo aleatorio con restricciones configurables



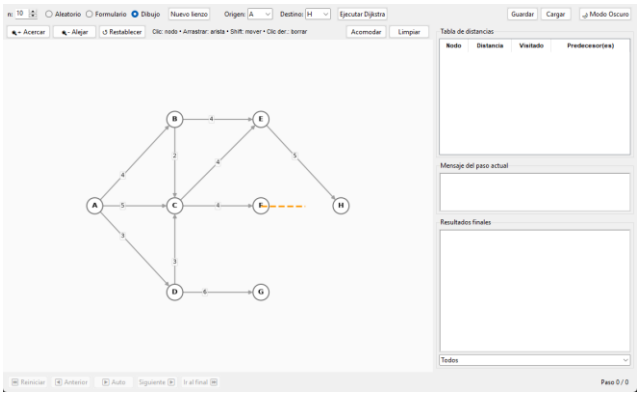
Nota. Barra de opciones del generador y resumen del grafo creado (origen A, destino H, mínimo de 4 aristas, empate forzado). Elaboración propia.

Modo de Dibujo Interactivo

Un clic en una zona vacía crea el siguiente nodo (A, B, C…) y arrastrar de un nodo a otro traza una arista con una línea guía; al soltar, se verifica con DFS que no cierre un ciclo y se solicita el peso (1 a 99). Con Shift se reubican los nodos y con clic derecho se eliminan nodos o aristas.

Figura 3

Dibujo interactivo del grafo con el mouse



Nota. Grafo de 8 nodos construido sobre el lienzo; la línea naranja discontinua es la arista F → H en trazado. Elaboración propia.

Búsqueda y Selección de Nodos

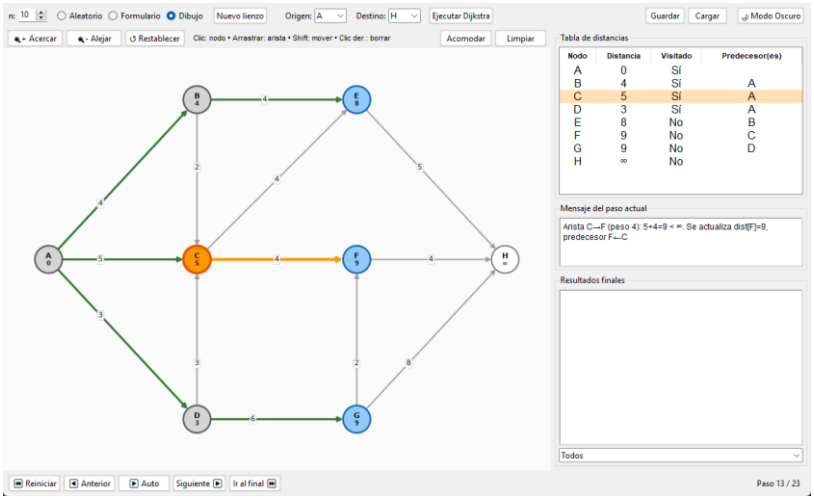
Una vez finalizado el grafo, o cargado desde un archivo JSON, las cajas de selección de la barra superior permiten escoger el nodo de origen A y de destino H para iniciar la ejecución del algoritmo.

Ejecución Interactiva del Algoritmo

Al pulsar Ejecutar Dijkstra, se activan los controles de navegación. El usuario puede avanzar paso a paso observando el nodo actual en color naranja, los nodos en espera en azul claro y la tabla de distancias actualizándose.

Figura 4

Ejecución interactiva paso a paso del algoritmo de Dijkstra

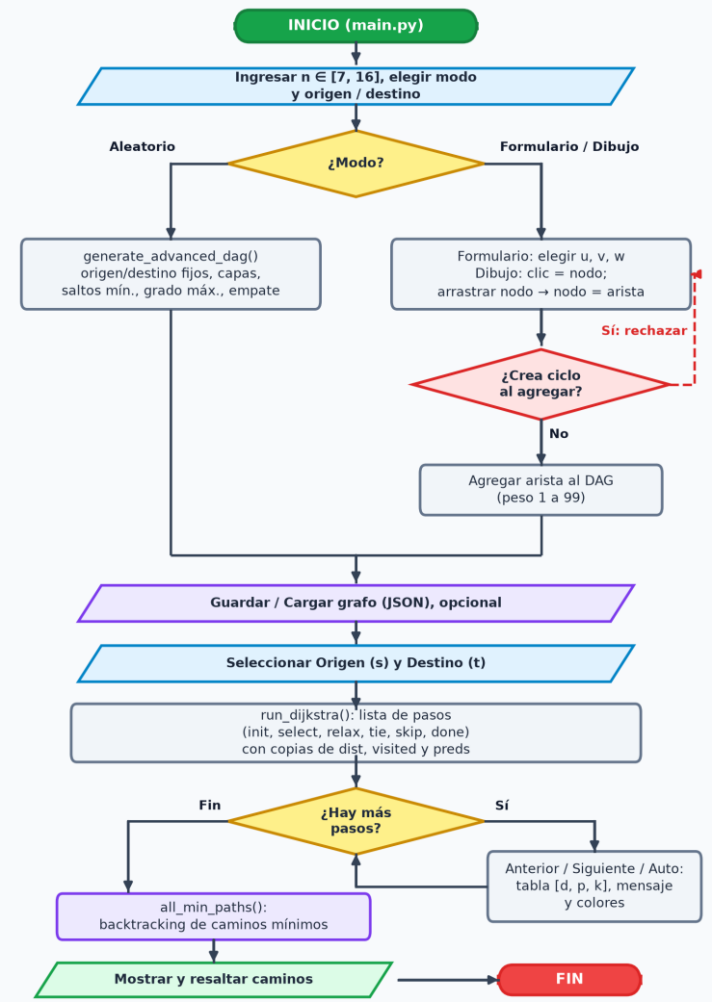


Nota. Paso 13 de 23: se evalúa el nodo C (distancia acumulada 5) y se relaja su arista hacia F (peso 4), con lo que dist[F] = 9. Elaboración propia.

Diagrama de flujo del programa ejecutable

Figura 5

Diagrama de flujo del sistema interactivo para el camino mínimo con Dijkstra



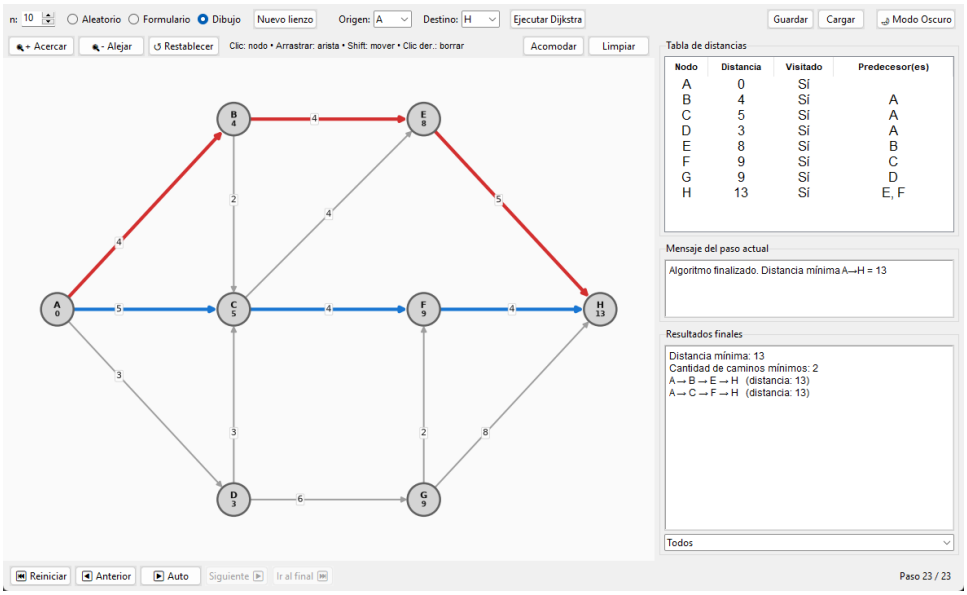
Nota. El diagrama muestra la secuencia de operaciones del programa: elección del modo, validación de aciclicidad, guardado y carga, ejecución interactiva y presentación de todos los caminos mínimos. Elaboración propia.

Visualización de Resultados

Al llegar al paso final, el panel derecho muestra los resultados con la distancia mínima encontrada, la cantidad de caminos mínimos y la lista de nodos de cada recorrido. En el grafo, las rutas óptimas se resaltan con líneas gruesas de colores (rojo para el primer camino y azul para el segundo), acompañadas de su respectiva leyenda cuando existen varios caminos.

Figura 6

Visualización de resultados finales y caminos mínimos resaltados



Nota. Pantalla final del programa: distancia mínima de 13 entre A y H y los 2 caminos mínimos encontrados (A → B → E → H y A → C → F → H), resaltados con colores. Elaboración propia.

Programa

El proyecto se programó en Python mediante una estructura modular compuesta por cinco archivos: main.py para iniciar la aplicación, graph_model.py para la estructura del grafo, la validación de ciclos, el generador con restricciones y el guardado en JSON, dijkstra.py para la lógica del algoritmo y el registro de pasos, visualizer.py para el dibujo con Matplotlib y los gestos del mouse, y ui.py para el diseño visual de la interfaz y los modos claro y oscuro. La aplicación permite pausar la ejecución automática, avanzar o retroceder pasos y seleccionar si se desean ver todos los caminos juntos o uno por uno. El repositorio se encuentra disponible en: https://github.com/TUXERO1986/Proyecto_dijkstra_Mate_Computacional

Resultados

Para comprobar el funcionamiento del programa, se realizaron pruebas experimentales sobre un banco de 10 instancias con tamaños de n = 7 a 16 nodos. En la prueba principal con 8 nodos (A hasta H), se configuró un grafo con 13 aristas y pesos entre 2 y 8. Al seleccionar A como origen y H como destino, el programa ejecutó 23 pasos de evaluación, encontrando que la distancia mínima es d* = 13. Además, identificó correctamente dos caminos mínimos distintos que empatan en costo: A → B → E → H (4 + 4 + 5 = 13) y A → C → F → H (5 + 4 + 4 = 13).

La ejecución paso a paso fue diseñada para apoyar la comprensión de la selección de nodos y la relajación de aristas. La literatura sobre visualización de algoritmos indica que estas herramientas pueden favorecer el aprendizaje cuando promueven una participación activa (Hundhausen et al., 2002). Durante las pruebas se comprobó que el entorno gráfico permite observar en qué momento se actualiza cada distancia y por qué se descartan otras opciones (como la ruta A → D → G → H, de costo 17); el algoritmo se resolvió en aproximadamente 0,07 ms por ejecución en Python 3.11, muy por debajo de 5 ms.

Para validar la exactitud, se resolvió a mano sobre papel un banco de 10 instancias (grafos de 7 a 16 nodos, generados de forma aleatoria y manual), anotando las ternas [dv, pv, k] y los predecesores paso a paso. Al compararlas con la salida del programa, los valores de distancia, los nodos visitados y los caminos mínimos finales coincidieron en un 100% de los casos. Además, las pruebas automáticas contrastaron Dijkstra con la enumeración exhaustiva de caminos en 150 grafos adicionales, con coincidencia total en distancia y cantidad de caminos óptimos.

Para validar el generador con restricciones se produjeron 1 000 grafos (n = 7 a 16, origen y destino aleatorios, mínimo de 4 aristas, máximo de 3 salidas por nodo y empate forzado). En el 100% de los casos el grafo fue acíclico, no existió ningún camino de menos de 4 aristas entre origen y destino, ningún nodo superó las 3 aristas salientes y se obtuvieron al menos dos caminos de costo mínimo idéntico; así, el generador evita los casos triviales señalados por el docente.

Observaciones del docente atendidas. En la primera revisión se recomendó evitar grafos aleatorios triviales y facilitar el ingreso del grafo dibujado a mano. Lo primero se resolvió con las restricciones del modo aleatorio (origen y destino fijos, saltos mínimos, máximo de conexiones por nodo, densidad, arista directa prohibida, empate y caminos alternativos). Lo segundo, con el modo de dibujo interactivo (Figura 3) y el guardado y carga en JSON, que permite abrir en segundos el grafo del informe.

Conclusiones

1. Cumplimiento del Objetivo: Se desarrolló una aplicación interactiva que permite crear (de forma aleatoria con restricciones, por formulario o dibujando) y analizar grafos dirigidos acíclicos de entre 7 y 16 nodos, aplicando el algoritmo de Dijkstra de forma visual y paso a paso, e incorporando las recomendaciones del docente.

2. Potencial Pedagógico: La interfaz con avance interactivo, tabla de distancias en tiempo real y mensajes descriptivos fue diseñada con potencial educativo; de acuerdo con Hundhausen et al. (2002), su valor formativo reside en propiciar una participación activa del usuario al explorar las decisiones del algoritmo.

3. Identificación de Múltiples Caminos: La inclusión de listas de predecesores y la búsqueda hacia atrás resolvieron los empates, identificando todos los caminos mínimos y resaltándolos con colores distintos; el generador puede, además, forzar estos empates para demostrarlo.

4. Integración Práctica: La combinación de Tkinter, Matplotlib y heapq demostró ser una solución eficiente para unir la teoría matemática con el desarrollo de software, creando un programa rápido, verificado con pruebas automáticas y fácil de usar para cualquier estudiante de ingeniería.

Referencias Bibliográficas

Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network flows: Theory, algorithms, and applications*. Prentice Hall.

Bondy, J. A., & Murty, U. S. R. (2008). *Graph theory*. Springer. https://doi.org/10.1007/978-1-84628-970-5

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). The MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*, 269–271. https://doi.org/10.1007/BF01386390

Gross, J. L., & Yellen, J. (2005). *Graph theory and its applications* (2nd ed.). Chapman & Hall/CRC. https://doi.org/10.1201/9781420057140

Hundhausen, C. D., Douglas, S. A., & Stasko, J. T. (2002). A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing*, *13*(3), 259–290. https://doi.org/10.1006/jvlc.2002.0237

Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, *5*(11), 558–562. https://doi.org/10.1145/368996.369025

Matplotlib Development Team. (2026). *Embedding Matplotlib in graphical user interfaces* (Matplotlib 3.11.2 documentation). https://matplotlib.org/stable/gallery/user_interfaces/index.html

NetworkX Developers. (2026a). *Directed acyclic graphs* (NetworkX 3.7 documentation). https://networkx.org/documentation/stable/reference/algorithms/dag.html

NetworkX Developers. (2026b). *Shortest paths* (NetworkX 3.7 documentation). https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html

Python Software Foundation. (2026a). *heapq—Heap queue algorithm* (Python 3.14.8 documentation). https://docs.python.org/3/library/heapq.html

Python Software Foundation. (2026b). *tkinter—Python interface to Tcl/Tk* (Python 3.14.8 documentation). https://docs.python.org/3/library/tkinter.html

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Skiena, S. S. (2020). *The algorithm design manual* (3rd ed.). Springer. https://doi.org/10.1007/978-3-030-54256-6

Universidad Peruana de Ciencias Aplicadas. (s. f.). *Lectura 5: Algoritmo de Dijkstra* [Material de curso]. Departamento de Ciencias. https://drive.google.com/file/d/1aLs_26jO-eGmSGPf6IsYw1_Q4HASBR6N/view